Applied Intelligence (2025) 55:784
https://doi.org/10.1007/s10489-025-06681-1

Multi-modal anomaly detection for microservice system through nested graph diffusion reconstruction

Mengwei Fan[1] · **Xiuguo Zhang**[1] · **Peipeng Wang**[1] · **Zhiying Cao**[1]

Accepted: 29 May 2025 / Published online: 19 June 2025
© The Author(s), under exclusive licence to Springer Science+Business Media, LLC, part of Springer Nature 2025

Abstract
Accurately detecting anomalies in microservice systems has become increasingly important for rapidly growing microservice architectures. On the one hand, existing anomaly detection methods typically focus on a single type of data source (i.e., logs or traces) or overlook the correlations between different types of data, which can lead to missed anomalies and a high number of false positives. On the other hand, regarding the issue of imbalanced positive and negative samples in anomaly detection, existing research has not proposed effective solutions. To address these issues, in this paper, we propose MSNGAD, a microservice anomaly detection method based on nested graph diffusion reconstruction. MSNGAD uses nested graphs to uniformly describe the complex structure of traces and embedded log messages. It treats the log template temporal relationships as the inner graph of the nested graph, and the service invocations relationships as the outer graph. By capturing the correlations between different modalities of data, the system can detect anomalies comprehensively. Furthermore, to address the problem of imbalanced positive and negative samples in anomaly detection, MSNGAD trains a diffusion reconstruction model to augment the features of anomalous samples. This approach abandons traditional manual data augmentation strategies, effectively mitigating the unnatural disturbances to the graph structure caused by human intervention in data augmentation. Extensive evaluations were conducted on large-scale datasets, and the experimental results show that MSNGAD outperforms existing baseline models, achieving an F1-score of 0.96.

Keywords Microservice · Anomaly detection · Nested graph · Feature augmentation · Deep learning

1 Introduction

With the widespread adoption of microservice architectures, the functionality of a single application is decomposed into multiple small services that operate independently, each operating in its own process and interacting through lightweight communication mechanisms. While this architecture enhances the system's flexibility and scalability, it also introduces new challenges, particularly in anomaly detection. Due to the intricate dependencies between microservices, an anomaly in one service can trigger a cascade of anomalies, leading to additional performance and reusability issues. Therefore, to enable operations engineers to detect potential anomalies in microservice systems promptly and take effective measures, automated anomaly detection is crucial.

In microservice systems, there are various types of monitoring data, such as logs and traces. Logs have been widely used in anomaly detection tasks and play a crucial role in software reliability engineering. With the widespread adoption of standards such as OpenTracing [1] and infrastructures such as SkyWalking [2], distributed traces have also been applied in anomaly detection tasks. As shown in Fig. 1, logs are semi-structured textual data, predefined by developers, used to record the various behaviors of different services in the system for debugging and root cause analysis. Traces describe the execution process (i.e., invocation chain) of a request through different service instances in a

✉ Xiuguo Zhang
zhangxg@dlmu.edu.cn

Mengwei Fan
fanmengwei@dlmu.edu.cn

Peipeng Wang
wpp7@dlmu.edu.cn

Zhiying Cao
czysophy@dlmu.edu.cn

[1] Information Science and Technology College, Dalian Maritime University, Dalian, Liaoning, China

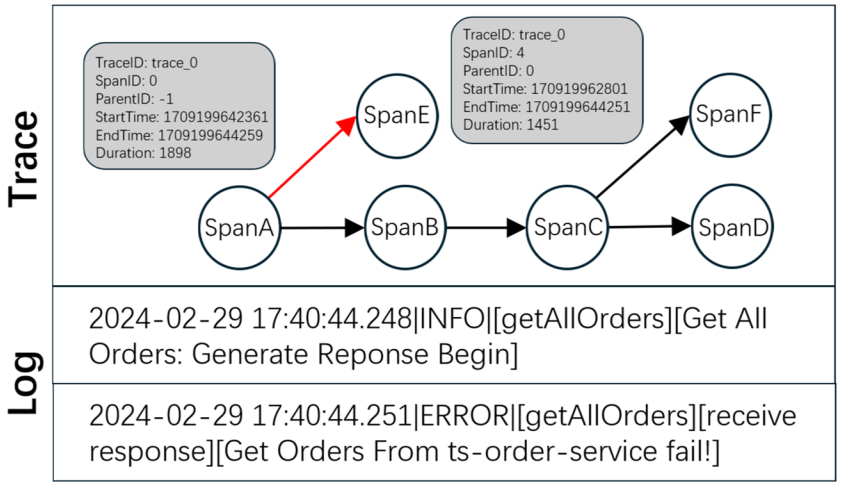


Fig. 1 A multi-modal data case

microservice, with each operation (i.e., service invocation) referred to as a span, and each span records various information related to the invocation.

Based on these monitoring data, most research uses deep learning methods to detect anomalies in microservice systems. For logs, existing log anomaly detection methods mainly rely on extracting semantic features from log texts [3–5] for anomaly detection. Many detection models have been developed by optimizing log parsing and feature processing methods. For traces, current mainstream anomaly detection methods [6–8] focus on detecting anomalies through the response time or invocation paths of microservices. However, due to the complexity and diversity of anomaly patterns in microservice systems, using a single source of monitoring data often fails to capture all abnormal behaviors within the microservices. For example, when a microservice is subjected to a malicious attack, the attacker may deliberately avoid detection, causing the corresponding node's logs to show no errors. However, analyzing the trace data can help identify such anomalies. Conversely, when there is an internal logic error in the service, the service may respond normally but return incorrect data. This type of anomaly cannot be detected from the traces but can be identified by inspecting the log information of the corresponding node. In summary, only by incorporating the analysis of logs and traces, and deeply analyzing the complex invocation relationships in microservices, can we comprehensively capture the abnormal behavior of the system.

Furthermore, in normal microservice systems, exceptions are usually rare. According to statistics, in a microservice system with millions of invocation requests per day, the occurrence rate of abnormal events due to network failures, server crashes, resource bottlenecks, etc., is typically less than 0.1%. This leads to the issue of imbalanced positive and negative samples in anomaly detection. Existing studies [8, 9] generally use fault injection tools to manually inject anomalies into microservices to generate more abnormal data, aiming to balance the ratio between positive and negative samples. Even so, this issue of class imbalance has not been completely resolved. For example, in the publicly available GAIA dataset in the field of Artificial Intelligence for IT Operations (AIOps), the positive-to-negative sample ratio after manually injecting anomalies is approximately 9: 1, meaning that only 1 out of 10 samples is abnormal, which still presents a class imbalance issue. To address this, Ying et al. [10] proposed a clustering-based self-training method aimed at improving KNN's performance on imbalanced log data. However, this method is overly dependent on data distribution and is sensitive to noise. TraceCRL [11] increases the number of anomalous traces using six manually designed anomaly simulation strategies (i.e., network delay injection and response code tampering). However, this method relies heavily on expert knowledge and is prone to introducing noise, leading to unnatural disturbances in the graph structure. Moreover, current data augmentation methods are mainly designed for single-modality data, and it is crucial to develop a more robust and generalizable augmentation method for multi-modal anomaly detection tasks.

To address the above problems, in this paper, we propose MSNGAD, a microservice anomaly detection method based on nested graph diffusion reconstruction. First, it uses Drain3 [12] to parse semi-structured log messages into log templates. For a given trace data, it obtains the log message templates that belong to the span in the trace based on service instance names and timestamps, aligning the two modalities of data. For log templates within a span, in order to effectively represent the execution order (i.e., temporal relationships) and dependency relationships (i.e., spatial relationships) during its generation, it models the log templates as a graph structure as the inner graph of the nested graph, uses the Graph Attention Network (GAT) [13] to learn the contextual relationships between the log templates in the inner graph. To better describe the complex hierarchical structure and dependency relationships between services

in the microservice system, it also models the trace based on the invocation relationship between service operations as the outer graph of the nested graph, uses the Graph Convolutional Network (GCN) [14] to learn the dependency relationships between service operations in the outer graph. In addition, to alleviate the imbalance problem between positive and negative samples, it uses a feature augmentation method based on diffusion reconstruction, which uses the encoder module of the Variational Autoencoder (VAE) to sample the latent features of the original anomalous data, and utilizes the trained diffusion model to augment the features, then uses the decoder module to reconstruct the augmented data to obtain the pseudo-anomaly samples. Finally, it relies contrastive learning to improve the quality and diversity of the generated pseudo-anomaly samples.

In summary, this paper makes the following contributions.

- We design microservice nested graphs to integrate and represent both traces and logs to dig deeper into the spatiotemporal dependencies between log templates nested within spans.
- We propose a feature augmentation model based on diffusion reconstruction, which not only addresses the imbalance issue in anomaly detection, but also alleviates the unnatural disturbance to graph structure information caused by traditional data augmentation methods.
- Extensive experiments on two real-world datasets show that the proposed MSNGAD model achieves competitive performance against all compared approaches, achieving 0.96 F1-score. Additionally, ablation studies further validate the effectiveness of each design in our model.

2 Related work

Currently, technologies such as deep learning and reinforcement learning have been widely applied across various fields, with their application scenarios continually expanding, driving the intelligent transformation of industries [15–20]. This paper studies the methods of AIOps for microservice systems using graph neural networks, comparing the limitations of single-modal data in anomaly detection and combining multi-modal data from traces and logs for comprehensive system anomaly detection.

2.1 Single-modal data anomaly detection

Log-based anomaly detection Log-based anomaly detection has been widely used in security applications, DeepLog [21] takes log template index sequence as input and uses the LSTM model to learn the sequential pattern of normal logs to detect anomalous logs that deviate from the normal pattern, but this method ignores the semantic information of the logs. In order to solve the problem, LogAnomaly [22] adds the semantic information of logs to the anomaly detection and uses the LSTM model to predict the next possible log result based on the historical log sequence, but if there is a loss or duplication of log data in the actual log system, it is likely to disrupt the sequential pattern of the logs, which makes the detection accuracy lower. Considering this problem, LogRobust [23] applies a Bi-LSTM model and an attention network to detect anomalies in log sequences, improving the robustness of anomaly detection. To capture dependencies between long distance logs. ContexLog [24] retains all log information and processes it using a classification model based on a Transformer encoder. Although the above methods analyze log sequences from both sequential and semantic perspectives, collaborative interactions between microservices mean that adjacent log messages not only have temporal continuity but may also originate from different service nodes. Therefore, ignoring spatial dependencies between logs may lead to insufficient analysis of log correlations, which in turn reduces the effectiveness of anomaly detection. To address this, LogGT [25] converts log sequences into a graph structure, considering the complex spatiotemporal dependencies of logs, and combines transfer learning methods for cross-system log analysis. But logs describe the behavior of service instances, they often fail to directly represent the invocation topology between services, leading to insufficient cross-service anomaly correlation analysis.

Trace-based anomaly detection In traditional trace anomaly detection methods, traces are typically treated as sequences. For example, Multimodal Trace [6] processes textual labels and response times in a trace into a sequential form and uses a multimodal LSTM to detect anomalies. TraceAnomaly [7] represents each trace using a service trace vector, considering both response time and invocation path features, and applies a Deep Bayesian Network to learn normal patterns and distinguish anomalous data. However, due to the highly dependent and complex nature of microservice systems, the complex spatial relationships between services are lost by considering traces as sequence information only, so existing research workers represent traces as graph structures and use graph neural networks to detect anomalies. TraceVAE [26] proposes a dual-variable graph VAE that uses structural VAE and temporal VAE to separately reconstruct the structural and temporal features of traces. PUTraceAD [27] constructs a span causal graph with node features such as operation names, response codes, and durations, and learns the network parameters using positive and unlabeled data. GRAND [28] detects anomalous behaviors in traces by

combining VAE and GAN. Although traces capture the global topology of service interactions, they lack the fine-grained semantic and temporal information of internal service logs, limiting the completeness of anomaly representation and affecting detection performance.

2.2 Multi-modal data anomaly detection

Compared to single-modal studies, multi-modal anomaly detection studies are relatively rare, but considering that single-modal data is difficult to comprehensively capture the anomalous behaviors of microservices, an appropriate combination of multi-modal data can effectively capture system anomalies. LogTraceAD [29] combines logs and traces to construct graphs and uses the Sentence Variational Autoencoder (S-VAE) to learn graph features. DeepTraLog [9] combines logs and each trace to construct a trace event graph, learns graph features using Gated Graph Neural Networks (GGNN), learns regular feature representations of the training data as well as minimized hyperspheres with deep SVDD (Support Vector Data Description), treats feature representations inside the hyperspheres as normal, and feature representations falling outside the hyperspheres as anomalies. However, when the abnormal behaviors of microservice systems are diverse, it is difficult to find suitable boundaries to identify abnormal samples in high-dimensional potential spaces.

3 Method

The objective of MSNGAD is to automatically and accurately detect microservice system anomalies using the combined data of logs and traces generated by microservices during operation. The overview of MSNGAD is presented in Fig. 2, which includes five steps. Data Preprocessing, Nested Graph Construction, Nested Graph Feature Extraction, Feature Augmentation, and Anomaly Detection.

3.1 Data preprocessing

Log The first step is to process the raw log messages. Logs, as semi-structured text data, record the event records generated at different time points during system operation. Each log record usually contains information such as timestamp, log level, message content and so on. To eliminate redundant information in logs, we use the Drain3 log parsing algorithm to extract structured log templates from raw log messages in a streaming manner, where each template represents a specific log event l and is assigned a unique ID.

Trace Trace as the textual data of invocation relationship between microservices consists of multiple span events and the causal relationship between them, each span event represents an operation of the service, and usually contains information such as trace ID, span ID, service instance name, start time, end time, and so on. In this paper, we define $Tr = \{tr_1, tr_2, \ldots, tr_n\}$. For a certain trace $tr_i = \{s_1, s_2, \ldots, s_p\}$, where s_i is a span event belonging to this trace.

Data alignment To align the logs and traces, we extract the log events belonging to the span during the execution of the microservice based on the service instance name, start time and end time involved in each span event $L^{s_i} = \{l_1, l_2, \cdots, l_q\}$, and embed them within the corresponding span.

3.2 Nested graph construction

Since there are obvious differences in the way logs and traces represent anomalies, with logs as text and traces as

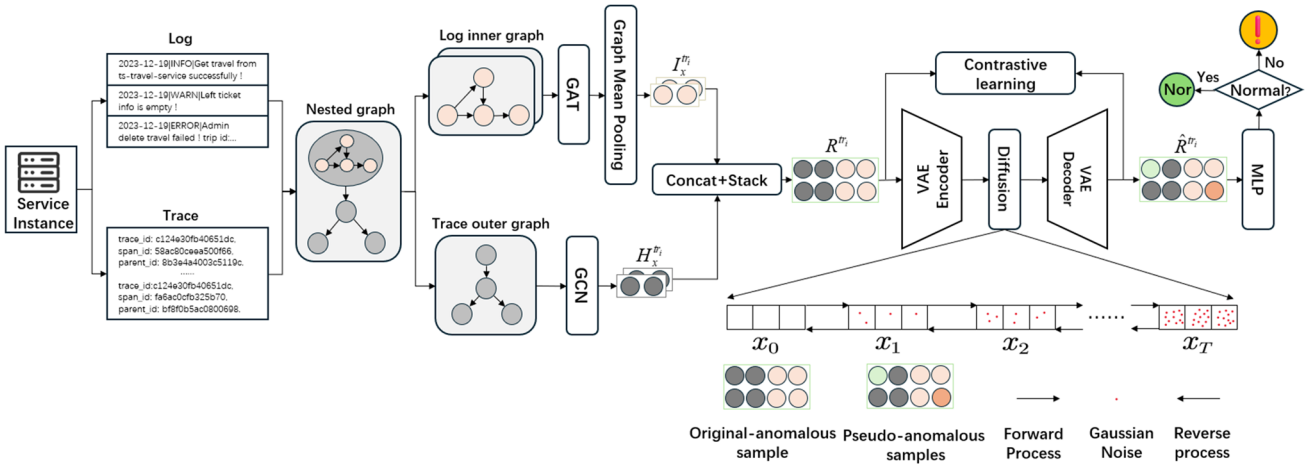


Fig. 2 MSNGAD overview

chain structures, this discrepancy poses a huge challenge to effectively combine the information for downstream anomaly detection, so how to effectively represent the correlation between the two becomes the focus of this study. Inspired by the nested graph structure, a trace can be regarded as graph structure information(outer graph), and the log events contained inside each span can also form a subgraph (inner graph), so we describe the complex dependencies between microservice invocations and the log events embedded in them by constructing nested graphs, and build nested graph inner graphs by log template timing relationships and nested graph outer graphs by service invocation relationships, effectively capturing the correlations between different modal data. The specific approach is as follows.

Inner graph The log template sequences within the trace span exhibit temporal dependencies due to the order of occurrence, and the same log may appear multiple times at different times. This suggests that certain operations or events in the system repeat in similar patterns at different points in time, indicating spatial dependencies as well, these dependencies reveal the distribution patterns of the same logs across different time points, reflecting the system's behavior patterns at various time stages. Therefore, in order to effectively analyze the spatiotemporal dependencies between log templates, MSNGAD models the log events inside each span as a graph structure, forming the inner graph layer of the nested graph $g^{s_i} = \{v^{s_i}, e^{s_i}\}$, where s_i denotes a certain span in the trace. In this graph, each node v^{s_i} represents different log events that belong to the span s_i, each edge e^{s_i} indicates the invocation order between different log events within the span s_i, the direction of the edge points from the calling log to the called log, and the frequency of node pairs appearing in the span is used as the weight on the edges. Specifically, as shown in Fig. 3, each node represents a different log event, for a certain log event $l_m^{s_i} \in L^{s_i}$, in order to better extract its semantic features, inspired by a large pre-trained language model, this paper uses a pre-trained BERT [30] language model to extract the features, log templates are processed by preprocessing [31], embedding layer and the transformer encoder, to generate template semantic features $z_m^{s_i}$, which are used as corresponding node features. using the execution order between different events in the log sequence as edges, and counting the number of times

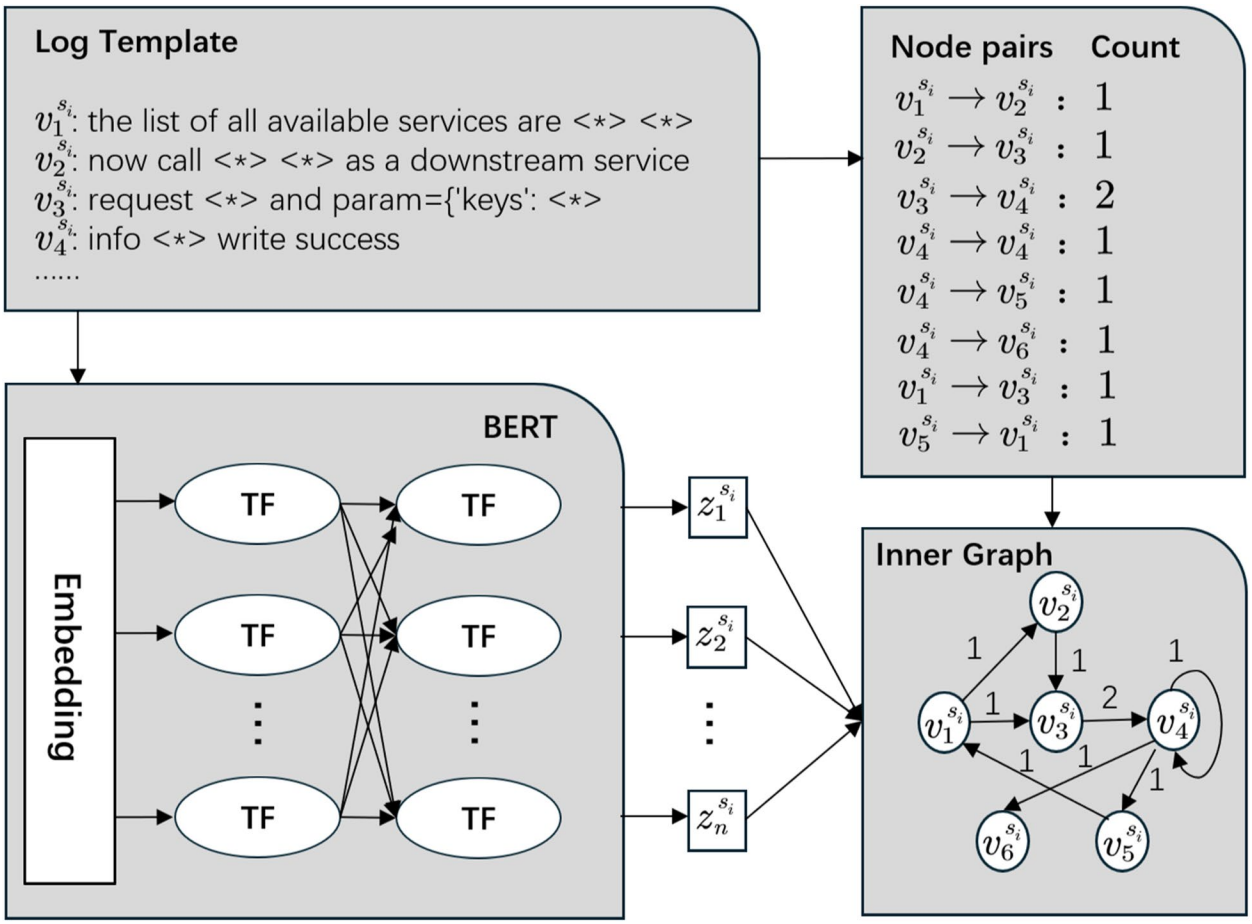


Fig. 3 Inner graph construction of the nested graph

the node pairs $v_m^{s_i} \longrightarrow v_n^{s_i}$ appear in that span, which are used as weights w_{mn} on the edges in the graph. Through this approach, the raw log messages are transformed into a directed, weighted graph containing multiple types of information, in which multiple implicit relationships of the system operation can be expressed more intuitively.

Outer graph To better describe the complex hierarchical structure and dependencies between service invocations in a microservice system, MSNGAD models span events as graph structures based on the contextual information recorded in the span and uses them as the outer graph layer of the nested graph $G^{tr_i} = \{V^{tr_i}, E^{tr_i}\}$. where tr_i denotes a certain trace. In the graph, each node V^{tr_i} represents different service operations within the trace tr_i, and each edge E^{tr_i} represents the invocation relationship between the service operations in the trace tr_i, with the direction pointing from a caller to a callee. For each node in the graph, we first calculate its initial feature using the time-related information recorded in the span, such as the start time and end time. Specifically, as shown in Fig. 4, suppose trace tr_i contains four spans, each recording the start and end times of the invocation. To better detect time-related anomalies in the system, we compute the following four time-related features as the initial node features.

- **Duration Time** is the duration of the span.
- **Waiting Time** is the waiting time of the callee for the response of the other services.
- **Local Execution Time** is the local execution time of the callee for the current invocation, excluding the waiting time.
- **Relative Start Time** is the difference between the start time of the current span and the start time of the root span.

Figure 4 illustrates an example of the calculation of the four time-related features. In the example, Span A is the root span and is the parent of Span B. Span B is the parent of Span C and Span D. The timestamps represent the start time and end time of each span. Assuming Span A is the root span, the duration time $d_t^A = t_8 - t_1$, waiting time $w_t^A = t_7 - t_2$, and local execution time $l_t^A = (t_2 - t_1) + (t_8 - t_7)$, since Span A is the root span, its relative start time $r_t^A = 0$. For Span C and Span D, since they have no child spans, the waiting time $w_t^C = 0$, $w_t^D = 0$, the duration time equals the local execution time, the relative start time of Span C is $r_t^C = t_3 - t_1$, and the relative start time of Span D is $r_t^D = t_5 - t_1$. Based on this, the time features $T_x^{tr_i} = [d_t^x, w_t^x, l_t^x, r_t^x]$, where $x \in \{A, B, C, D\}$, each span in the trace tr_i can be calculated and used as the initial feature vector of the nodes in the outer graph.

3.3 Nested graph representation

Inner graph features Considering that different neighboring nodes in the inner graph have varying levels of importance, as shown in Fig. 5, we use GAT to dynamically assign weights to the relative importance of each neighbor node. The GAT adaptively learns the importance of each neighboring node, capturing the complex relationships between nodes. For the inner graph $g^{s_i} = \{v^{s_i}, e^{s_i}\}$, $z_m^{s_i}$ represents the feature vector of the current node $v_m^{s_i} \in v^{s_i}$, and the output representation $h_m^{s_i}$ of each node is obtained through the GAT network. The definition of the GAT is given by the (1).

$$h_m^{s_i} = \sigma\left(\sum_{n \in N(m)} \alpha_{mn} W z_m^{s_i}\right) \tag{1}$$

where $N(m)$ represents the set of neighboring nodes of node m, $W \in R^{F' \times F}$ is the weight matrix to be learned, and α_{mn} is the normalized attention weight between node m and node n, which is calculated by the (2).

$$\alpha_{mn} = \frac{\exp\left(\text{Leaky ReLU}\left(a^T\left[Wh_m^{s_i} \,||\, Wh_n^{s_i}\right]\right)\right)}{\sum_{k \in \mathrm{N}(m)} \exp\left(\text{Leaky ReLU}\left(a^T\left[Wh_m^{s_i} \,||\, Wh_k^{s_i}\right]\right)\right)} \tag{2}$$

where W and a are learnable parameters, and $Leaky\mathrm{ReLU}(\bullet)$ represents the activation function. For

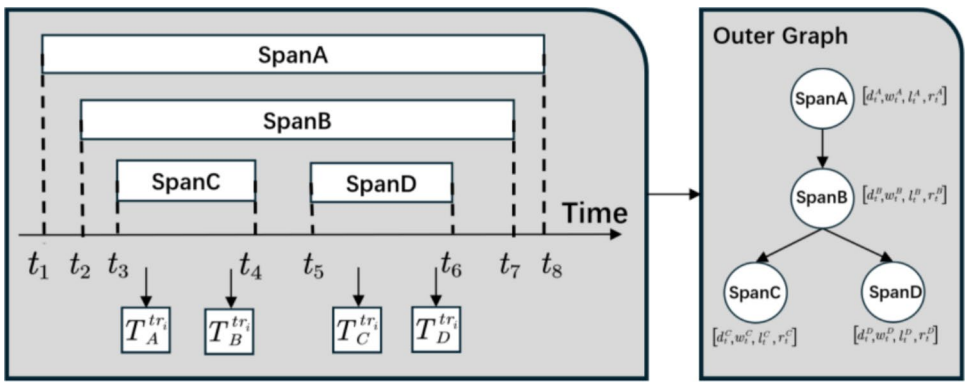


Fig. 4 Outer graph construction of the nested graph

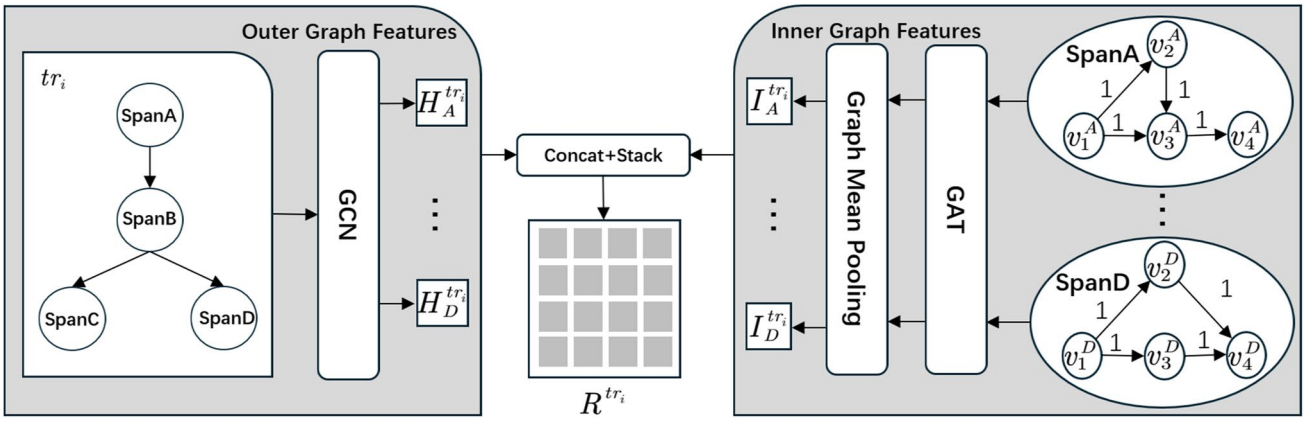


Fig. 5 Illustration of nested graph feature extraction

node m, the output feature $h_m^{s_i}$ is obtained through GAT, and then, by applying an average pooling operation, the global graph vector I^{s_i} is obtained, which is used as the log feature part $I_x^{tr_i}$ of the corresponding node x in the outer graph.

Outer graph features To more accurately capture the information flow between nodes, we use GCN to normalize the number of neighbors for each node in the outer graph, all the model to focus more on nodes with higher neighbor density. This helps effectively capture the potential dependencies between nodes. Taking the outer graph in Fig. 5 as an example, for a given node x in the outer graph, the feature representation $H_x^{tr_i}$ of each node is obtained after applying a single layer GCN as (3).

$$H_x^{tr_i} = \mathrm{ReLU}\left(\tilde{D}_x^{-\frac{1}{2}} \tilde{A}_x^{tr_i} \tilde{D}_x^{-\frac{1}{2}} T_x^{tr_i} W_x\right) \tag{3}$$

where $\tilde{A}_x^{tr_i} = A_x^{tr_i} + I$ is the matrix obtained by adding the identity matrix I to the adjacency matrix $A_x^{tr_i}$ and normalizing it, ensuring that the GCN can learn the node's own features. $\tilde{D}_x$ is the degree matrix of $\tilde{A}_x^{tr_i}$. The graph convolution operation achieves the aggregation of normalized neighbor information, and $\tilde{D}_x^{-\frac{1}{2}} \tilde{A}_x^{tr_i} \tilde{D}_x^{-\frac{1}{2}}$ represents the normalized adjacency matrix. $T_x^{tr_i}$ is the feature matrix of node $V_x^{tr_i}$, W_x is the trainable weight matrix in the GCN, and $\mathrm{ReLU}(\bullet)$ is the activation function.

Feature fusion The feature representation $H_x^{tr_i}$ of each node is concatenated with the log features $I_x^{tr_i}$ within the span to obtain the updated node representation $Z_x^{tr_i}$, which is calculated by the (4).

$$Z_x^{tr_i} = \mathrm{Concat}\left(H_x^{tr_i}, I_x^{tr_i}\right) \tag{4}$$

where $\mathrm{Concat}(\bullet)$ denotes the concatenation operation. Then, aggregate all node representations to obtain the updated graph features R^{tr_i}, which is calculated by the following (5).

$$R^{tr_i} = \mathrm{Stack}\left(Z_1^{tr_i}, Z_2^{tr_i}, \ldots, Z_p^{tr_i}\right) \tag{5}$$

3.4 Feature augmentation

Traditional graph data augmentation methods [32, 33] mostly rely on manual modifications of graph features and structures, requiring deep expertise and often introducing noise [34]. Although Generative Adversarial Networks (GANs) [35] have strong generative capabilities, they are prone to mode collapse when anomaly samples are scarce, resulting in insufficient diversity in the generated samples. Mixup [36, 37] generates new samples through linear interpolation, but due to the sparse and discrete nature of anomaly samples, key anomaly features may be diluted during mixing, weakening the anomaly characteristic. In contrast, diffusion models learn the underlying distribution of the data through gradual noise addition and removal, reducing manual intervention while avoiding mode collapse. Additionally, diffusion models can control the extent of augmentation at different noise levels, enriching the data distribution and improving anomaly detection performance. The specific approach is as follows.

VAE encoder To ensure the diversity of the generated pseudo-anomaly samples, it is far from sufficient to only augment the diffusion of the original input data. Considering the working principle of the VAE, the corresponding hidden variables are computed by sampling the distributions in the latent space using the reparameterization technique, in this way, the model can introduce randomness into the hidden variables, so that the samples fed into the diffusion model have a certain degree of randomness. In MSNGAD, the VAE encoder module maps the graph feature R^{tr_i} to the

latent space, and by sampling from the distribution in this latent space, it generates latent variables x^{tr_i} with inherent randomness, which is calculated by the (6).

$$x^{tr_i} = \mu + \varepsilon \times \sigma \tag{6}$$

where μ and σ represent the mean and standard deviation of the latent space, and $\varepsilon \sim N(0, I)$ is random noise, with N denoting the normal distribution and I being the identity matrix. This noise, combined with the mean and standard deviation using the reparameterization trick, is applied to avoid the issue where the sampling operation cannot participate in the gradient descent.

Forward process For the low-dimensional vector x^{tr_i} obtained through sampling, Gaussian noise is progressively added through the forward diffusion process. In this process, at each time step, the information x_t is only dependent on the previous step x_{t-1}. The forward process is described by the (7).

$$q\left(x_t^{tr_i} \middle| x_{t-1}^{tr_i}\right) = N\left(x_t^{tr_i}; \sqrt{1-\beta_t} x_{t-1}^{tr_i}, \beta_t I\right) \tag{7}$$

where $\beta_t \in (0, 1)$ represents the intensity of the Gaussian noise added at step t, which increases as the time step increases. To control the noise intensity variation at each step, we refer to previous work [34], we use linear noise control to regulate the noise intensity. This is calculated by the (8).

$$\beta_t = k \times \left(\alpha_{\min} + \frac{t-1}{T-1}\left(\alpha_{\max} - \alpha_{\min}\right)\right), \quad t \in \{1, 2, \ldots, T\} \tag{8}$$

where $\alpha_{\min} < \alpha_{\max} \in (0, 1)$ are the minimum and maximum values of the noise, $k \in [0, 1]$ is a hyperparameter that adjusts the noise intensity, and T is the total number of forward steps.

Reverse process After the noise addition process ends and pure noise $x_T^{tr_i}$ is obtained, the diffusion model's reverse process is used to remove the noise, gradually restoring the original vector from $x_T^{tr_i}$. The reverse process is described by the equation (9).

$$P\left(x_{t-1}^{tr_i} \middle| x_t^{tr_i}\right) = N\left(x_{t-1}^{tr_i} \middle| \mu\left(x_t^{tr_i}, t\right), \sigma\left(x_t^{tr_i}, t\right)\right) \tag{9}$$

where $\mu\left(x_t^{tr_i}, t\right)$ and $\sigma\left(x_t^{tr_i}, t\right)$ are the mean and variance used to recover the state $x_{t-1}^{tr_i}$ from the previous state $x_t^{tr_i}$.

VAE decoder After generating augmented features through the denoising process of the diffusion model, the decoder part of the VAE is used to reconstruct and generate augmented features $\hat{R}^{tr_i}$ that are similar to the original features.

Contrastive learning To ensure the quality of the generated pseudo-anomaly samples, this paper introduces the idea of contrastive learning. In the feature space, the distance between the pseudo-anomaly sample $\hat{R}^{tr_i}$ and the real anomaly sample R^{tr_i} is minimized, while the distance from the normal sample R^{tr_j} is maximized. This ensures that the generated pseudo-anomaly samples retain the original anomaly characteristics, as shown in the (10).

$$L_c = -\frac{1}{N}\sum_{i=1}^{N} \log \frac{\exp\left(\operatorname{sim}\left(\widehat{R}^{tr_i}, R^{tr_i}\right)/\tau\right)}{\exp\left(\operatorname{sim}\left(\widehat{R}^{tr_i}, R^{tr_i}\right)/\tau\right) + \sum_{j \neq i} \exp\left(\operatorname{sim}\left(\widehat{R}^{tr_i}, R^{tr_j}\right)/\tau\right)} \tag{10}$$

where L_c is the contrastive loss, τ is the temperature coefficient, and $sim(\bullet)$ is the similarity function used to control the similarity between pseudo-anomaly samples, normal samples, and anomalous samples, ensuring the quality and effectiveness of the augmented samples.R^{tr_j} represents the normal samples in the same batch.

However, since system anomalies are influenced by multiple factors and exhibit randomness and complexity, attempting to further reduce the spatial distance between the pseudo-anomaly and anomaly samples may lead the model to fit specific anomalies, ultimately reducing its generalization ability. Therefore, to further ensure the diversity of pseudo-anomaly samples, upper and lower threshold bounds are set in the feature space. This ensures that the similarity between the generated pseudo-anomaly samples and the real anomaly samples remains high (i.e., above the lower bound) but not identical (i.e., not exceeding the upper bound). In this way, the pseudo-anomaly samples can accurately capture the original anomaly features while remaining distinct, thereby ensuring the diversity of the generated anomaly samples. Based on this, we define the margin constraint loss, as shown in the (11).

$$\begin{aligned} L_{\text{margin}} = {} & \lambda_1 \max\left(0, \text{sim}_{\text{low}} - \text{sim}\left(\hat{R}^{tr_i}, R^{tr_i}\right)\right) \\ & + \lambda_2 \max\left(0, \text{sim}\left(\hat{R}^{tr_i}, R^{tr_i}\right) - \text{sim}_{\text{high}}\right) \end{aligned} \tag{11}$$

where λ_1 and λ_2 are the weight parameters for each term, sim_{low} and sim_{high} represent the lower and upper bounds of similarity, respectively.

Loss We use three loss functions to jointly optimize the diffusion reconstruction model: the VAE model loss, the diffusion model loss, and the contrastive learning loss. The loss is defined by the (12).

$$L_i = (\mathrm{MSE}\left(R^{tr_i}, \hat{R}^{tr_i}\right) + D_{KL}\left(q\left(x^{tr_i} \mid R^{tr_i}\right) \| p\left(x^{tr_i}\right)\right)) + \left\|\varepsilon_i - \hat{\varepsilon}_i\left(x_t^{tr_i}, t\right)\right\|_2^2 + (L_c + L_{\text{margin}}) \tag{12}$$

where L_i is the total loss of sample i in the diffusion reconstruction module, $MSE(\bullet)$ and $D_{KL}(\bullet)$ together form the loss of the VAE model, $MSE(\bullet)$ represents the mean squared error between R^{tr_i} and $\hat{R}^{tr_i}$, used to calculate the reconstruction error of the VAE. $D_{KL}(\bullet)$ represents the KL divergence, where $q\left(x^{tr_i} \mid R^{tr_i}\right)$ is the approximate posterior distribution used to learn the distribution of latent variables from input data R^{tr_i}, $p(x^{tr_i})$ is the prior distribution, which is the assumed distribution of latent variable x^{tr_i}, and in this paper, the standard normal distribution $N(0, I)$ is chosen. ε_i and $\hat{\varepsilon}_i(x_t^{tr_i}, t)$ refer to the true noise and predicted noise of sample i at time step t, respectively. $\| \bullet \|_2$ represents the L2 norm, used to calculate the loss between the true noise and the predicted noise. $(L_c + L_{\text{margin}})$ represents the contrastive loss with the added margin constraint.

3.5 Anomaly detection

In the anomaly detection phase, we train an MLP classification network with a $softmax$ output layer to predict the probabilities $\hat{y}$ of normal and anomalous states. The probabilities $\hat{y}$ are calculated by the (13).

$$\hat{y} = \mathrm{softmax}\left(W R^{tr_i} + b\right) \tag{13}$$

where W and b are the weight matrix and the bias term of the output layer. During training, the model's predictions are compared with the true labels to compute the cross-entropy loss, which is used to optimize the model's prediction performance. For a given new trace and its corresponding logs, MSNGAD follows the same process as in the training phase to generate the nested graph. It uses GAT and GCN to extract features from the inner and outer graphs, respectively. The fused features are then passed to the trained classification network to detect anomalies, Algorithm 1 outlines the complete learning process of MSNGAD.

Algorithm 1 The learning process of MSNGAD

Input: Traces T, Logs L, number of epochs I
Output: Abnormal detection results: Y
1: Data preprocessing: trace scheduling relationship extraction, log parsing, log semantic encoding, multi-modal data alignment
2: **for** epoch ← 1 to I **do**
3: Construct the inner graph g^{s_i} and outer graph G^{tr_i} of the nested graph;
4: Inner graph feature representation $I_x^{tr_i} = MeanPool(GAT(g^{s_i}))$;
5: Outer graph node feature representation $H_x^{tr_i} = GCN(G^{tr_i})$;
6: Nested graph feature representations $R^{tr_i} = Stack(Concat(H_x^{tr_i}, I_x^{tr_i}))$;
7: Obtain pseudo-anomaly samples $\hat{R}^{tr_i}$ through diffusion reconstruction;
8: Compute VAE model loss, diffusion model loss, and contrastive learning loss;
9: Update model parameters by minimizing (12);
10: **end for**
11: Obtain anomaly detection results;
12: **return** Y;

4 Evaluation

In our study, we evaluate the MSNGAD method and discuss the following questions:

- **RQ1:** How effective is MSNGAD in microservice anomaly detection compared to existing baseline methods, including single-modal and multi-modal methods?
- **RQ2:** How do different experimental parameters in MSNGAD affect anomaly detection results?
- **RQ3:** How effective are the different modules in MSNGAD?

4.1 Experimental design

4.1.1 Dataset

Our experimental studies are conducted based on two datasets: TrainTicket (TT) and Generic AIOps Atlas (GAIA)[1].

TT is a dataset collected from deploying TrainTicket on a Kubernetes cluster. It is a medium-sized open-source microservice system for booking train tickets, which has been widely applied in microservice architecture,

[1] https://github.com/CloudWise-OpenSource/GAIA-DataSet

infrastructure, and AIOps related research [38, 39]. The dataset includes 41 microservices written in 45 different languages, including Java, JavaScript, and Python, which communicate via synchronous REST calls and asynchronous messaging. We deployed TT on a Kubernetes cluster with 4 virtual machines, each configured with 8 CPU cores and 32GB of memory. The cluster also hosts the TT application, ElasticSearch [40], and Apache SkyWalking [2]. We used Python scripts to automatically send requests to the deployed microservice system, simulating user requests in various scenarios. Apache SkyWalking was used as the distributed tracing framework to collect trace and log information, while ElasticSearch was used to store the collected data. During data collection, we used fault injection methods to collect abnormal data. Following previous works [41–43], this experiment used the widely used chaos engineering tool, chaos-mesh [44], to inject network failures (i.e., network loss, network delay) and pod failures (i.e., CPU pressure, memory pressure, request loss) into different services separately.

GAIA is an open dataset in the field of AIOps, provided by CloudWise. It is collected from a simulated microservice system environment consisting of 5 hosts and 10 microservices, including redisservice1, webservice1, loginservice1, etc. This dataset contains over 2 million trace records and 87 million log records. Additionally, this dataset simulates real-world failure scenarios by injecting system failures (i.e., system freeze, process crash) and service failures (i.e., login failure, file loss, access denied).

More detailed information is shown in Table 1, where templates represent the number of log templates obtained by parsing logs with Drain3.

4.1.2 Baseline approaches

In our study, we compare the MSNGAD method with the following baseline approaches, which include both single-modal log and trace anomaly detection methods, as well as multi-modal anomaly detection methods.

- **DeepLog** [21] is a log anomaly detection method that takes log template index sequences as input. It uses an LSTM model to learn the sequential patterns of normal logs and detects anomalous logs by identifying deviations from the normal pattern.

Table 1 The details of two datasets

Dataset	Services	Templates	Records	%Failures
TT	41	294	Trace: 121,280 Log: 1,359,061	2.3%
GAIA	10	43	Trace: 2,321,280 Log: 87,974,577	8.9%

- **LogRobust** [23] is a log anomaly detection method that employs the FastText algorithm to model the semantic representation of log templates. It combines the TF-IDF algorithm to generate semantic vectors for logs and uses an attention-based Bi-LSTM model for anomaly detection.
- **TraceAnomaly** [7] is a trace anomaly detection method that represents traces as service trace vectors and trains a VAE model to detect anomalous traces.
- **TraceVAE** [26] is a trace anomaly detection method that uses a dual-variable graph VAE to separately encode and reconstruct temporal and structural features of traces. Anomalies are detected by calculating the reconstruction score.
- **DeepTraLog** [9] is a multi-modal anomaly detection method for microservices that combines logs and each trace data to construct trace event graph. It uses deep support vector data description to train a one-class classification model for anomaly detection.

4.1.3 Evaluation metrics

This paper uses Precision, Recall and F1-score as evaluation metrics to measure the performance of anomaly detection, defined as follows: $Precision = \frac{TP}{TP+FP}$, $Recall = \frac{TP}{TP+FN}$, $F1-score = \frac{2\times Precision\times Recall}{Precision+Recall}$, where TP represents the number of instances correctly classified as anomalies by the model, FP represents the number of instances incorrectly classified as anomalies, and FN represents the number of undetected anomalies.

4.1.4 Experimental settings

We implement MSNGAD using PyTorch 2.5.3, Python 3.9.19, and PyTorch Geometric 2.5.3. During the experiments, 70% of the total data is used for training, and 30% is reserved for testing. In the data collection process, the data generated by the system during the time period when faults are injected is considered anomaly data, while other data is considered normal data generated during system operation. The settings of MSNGAD are the following: use the GAT to extract the nested inner graph features, set the number of layers of the GAT to 2, set the dimensionality of the output features to 16, and use the GCN to extract the nested outer graph features, set the number of layers of the GCN to 2, and the output feature dimension set to 4, and the batch size is set to 64, feature augmentation ratio factor in each batch is set to 3, and the noise scale of the diffusion model is set to $1e^{-4}$. MSNGAD uses Adam as the optimizer, epoch is set to 20, and the learning rate is set to 0.001. For the

baseline approaches, this paper uses their publicly available implementation, preprocesses the experimental data, and configures the model based on the parameters in the original paper, the final experimental results are obtained by averaging over several experiments.

4.2 RQ1: effectiveness of MSNGAD

Table 2 shows the effectiveness evaluation results of MSNGAD and various baseline approaches on two different datasets. The results indicate that MSNGAD outperforms all baseline approaches. On both datasets, its Precision, Recall and F1-score achieved values of 0.939, 0.978, 0.958 and 0.941, 0.982, 0.961, respectively. With a high Recall, MSNGAD demonstrates its ability to accurately detect most anomalies in microservices, with very few false positives.

Among the two log-based anomaly detection methods, DeepLog shows poorer performance. The reason lies in the fact that DeepLog only uses LSTM to learn the sequence patterns of normal logs. However, the relationships between logs in a microservices system are complex, and relying solely on a single data feature for accurate prediction is challenging, which results in a noticeable decline in DeepLog's detection performance. In contrast, LogRobust improves the detection results as it not only considers the sequence patterns of normal logs but also utilizes semantic vectors of logs as features for detection. This enables it to effectively learn the distinguishing semantic features between normal and abnormal data in the log sequence. However, logs generated during the operation of a microservices system have complex dependencies. Treating them only as simple time-series data may overlook the spatial dependencies between logs. Moreover, both methods do not consider system trace information, limiting the reflection of the anomaly detection results.

Among the trace-based anomaly detection methods, TraceVAE outperforms TraceAnomaly by 15.6% and 22% in Recall, respectively, for the following reasons: First, TraceAnomaly constructs service trace vector that only reflects the sequential relationship between services in the trace, failing to fully capture the causal relationships between spans. In contrast, TraceVAE uses a dual-variable graph approach, leveraging graph structures to represent the complex relationships in the trace. Secondly, TraceAnomaly focuses primarily on the response time of trace and therefore can only detect anomalies that significantly impact the service response time distribution. But TraceVAE reconstructs both time and spatial features using the dual-variable graph, considering multiple aspects of the trace data and thus detecting more anomalies. Despite this, its performance still lags behind MSNGAD, mainly because it lacks log data and cannot detect abnormal behaviors within log events.

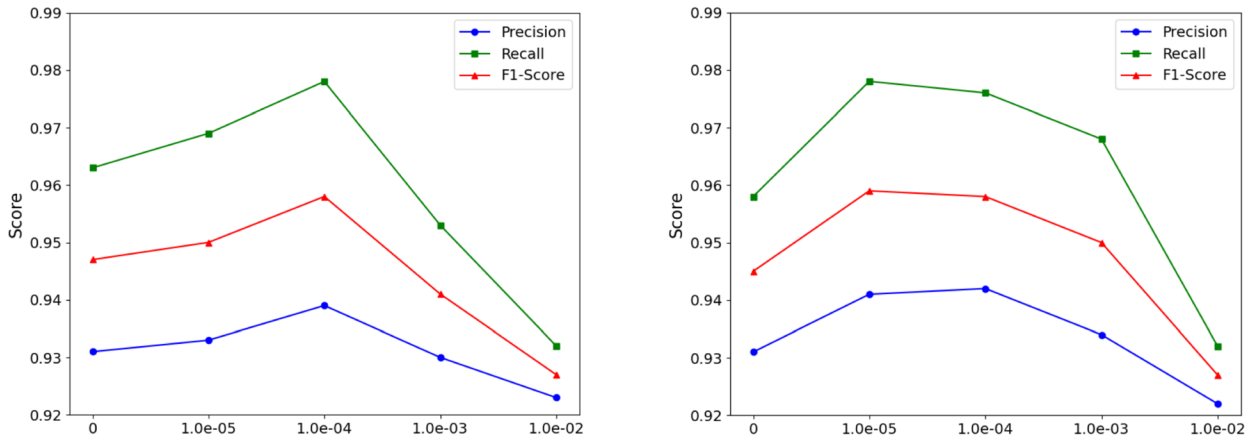
Among all baseline approaches, DeepTraLog achieved the best anomaly detection results. This is because DeepTraLog combines log and trace data, using a unified graph representation to describe the complex trace structure and embedded log events. Through feature interactions between different modalities, DeepTraLog can fully exploit the connections between anomalous behaviors across modalities. However, MSNGAD outperforms DeepTraLog by 1.8% and 2.0% in F1-score on the two datasets, respectively. The reason for this is that, in anomaly detection, DeepTraLog uses Deep SVDD to learn the conventional feature representations of the training data and minimize the hypersphere. However, when anomalies in a microservices system are highly diverse, it is challenging to find an appropriate boundary to identify anomalous samples in the high-dimensional latent space. In contrast, MSNGAD uses feature augmentation to increase the diversity of anomalous behaviors, enabling the model to see more anomalous samples during the training phase and thus detect more diverse anomalies.

4.3 RQ2: impact of hyperparameters

To further explore the impact of different hyperparameter settings on anomaly detection performance, this paper conducts a hyperparameter sensitivity experiment. Specifically, we focus on the sensitivity of two important hyperparameters in the feature augmentation module: the noise scale k and the feature augmentation ratio factor n. When analyzing each specific parameter, we keep all other parameters constant. We use Precision, Recall and F1-score as evaluation metrics to assess the experimental performance under different hyperparameter settings and perform experiments on both the TT and GAIA datasets to comprehensively evaluate

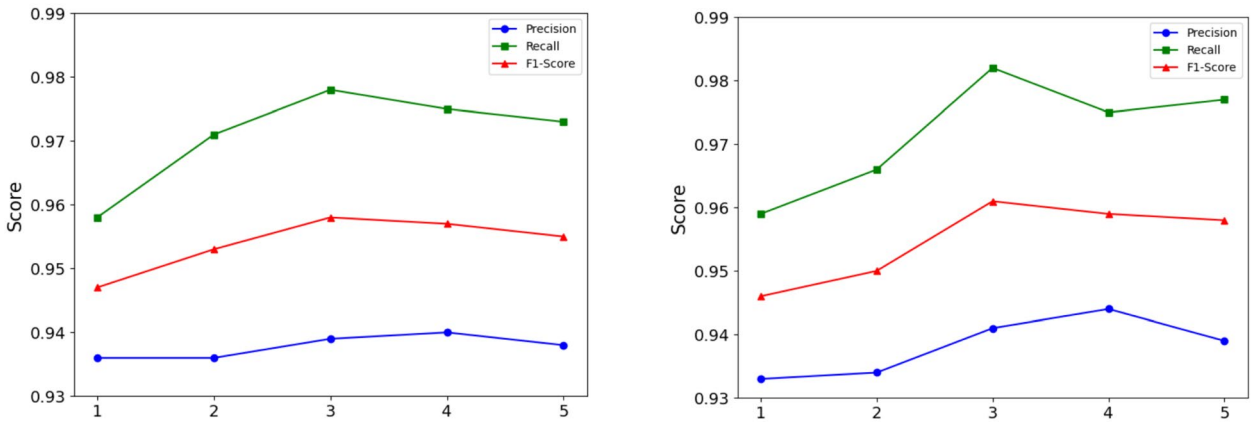
Table 2 Experimental results: best results are in bold font

Approach	TT			GAIA		
	Precision	Recall	F1-score	Precision	Recall	F1-score
DeepLog	0.426	0.719	0.535	0.411	0.682	0.512
LogRobust	0.766	0.817	0.789	0.707	0.811	0.755
TraceAnomaly	0.502	0.673	0.575	0.539	0.637	0.584
TraceVAE	0.619	0.892	0.731	0.756	0.859	0.804
DeepTraLog	0.933	0.948	0.940	**0.947**	0.935	0.941
MSNGAD	**0.939**	**0.978**	**0.958**	0.941	**0.982**	**0.961**



(a) k on TT (b) k on GAIA

Fig. 6 Sensitivity experiment of noise scale k



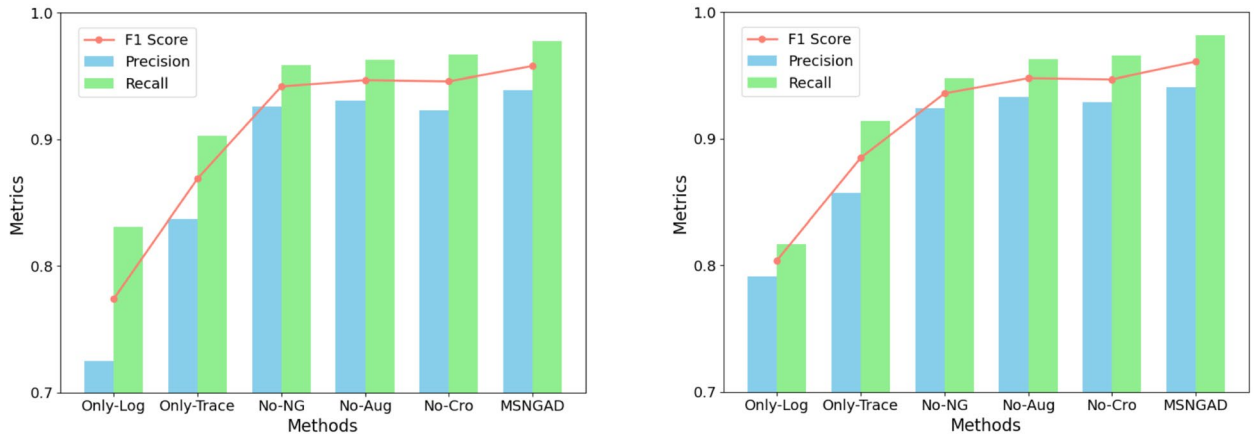
(a) n on TT (b) n on GAIA

Fig. 7 Sensitivity experiment of feature augmentation ratio factor n

the changes in model performance under different hyperparameter configurations.

Noise scale k In the experiment on the sensitivity of the noise scale hyperparameter k, we evaluate k values of 0, $1e^{-5}$, $1e^{-4}$, $1e^{-3}$ and $1e^{-2}$. The experimental results are shown in Fig. 6. As can be seen, with the increase in noise scale, the performance shows a trend of first increasing and then decreasing. Compared to the baseline case with no noise ($k = 0$), a moderate amount of noise (such as $k = 1e^{-4}$) can improve the model's generalization ability and enhance its learning of anomaly patterns, further validating the effectiveness of noise perturbation and denoising training. However, when k increases further, the model performance drops significantly because excessive noise may disrupt the original graph structure, making it difficult for the model to extract valid features during training, thus leading to a decline in the quality of generated samples. Therefore, we choose a relatively small noise scale of $1e^{-4}$ as the optimal setting for feature augmentation.

Feature augmentation ratio factor n In the experiment on the impact of the feature augmentation ratio factor n, as shown in Fig. 7, we observe the effect on anomaly detection when n takes values of 1, 2, 3, 4, and 5. When $n < 3$, the anomaly detection performance improves as n increases, indicating that moderately increasing the diversity of pseudo-anomaly samples helps the model learn more generalized anomaly patterns. However, when $n > 3$, the anomaly detection performance starts to decline. This is because excessive augmentation data might lead the model to overly rely on these pseudo-anomaly samples. If these samples do not reflect actual anomaly patterns, they negatively affect the model's learning process, thereby reducing the model's ability to identify real anomalies. Based on this, we select $n = 3$ as the optimal setting for the feature



(a) Ablation Experiment on TT (b) Ablation Experiment on GAIA

Fig. 8 Ablation experiment

augmentation ratio factor, ensuring the validity of the augmented data while avoiding the negative effects of excessive augmentation.

4.4 RQ3: effectiveness of different modules

To further verify the effectiveness of each module, we conducted several ablation experiments on MSNGAD. Specifically, the following modifications were made to the original model:

(1) Only-Trace: The log data is removed, and only the trace data is retained.
(2) Only-Log: The trace data is removed, and only the log data is retained.
(3) No-NG: The nested graph structure is removed, and only the BERT-extracted log features are used as the update features for the external graph nodes.
(4) No-Aug: The feature augmentation module is removed.
(5) No-Cro: The feature augmentation module is retained, but the contrastive learning is removed.

The above five variant models were experimented on the datasets TT and GAIA respectively, and the experimental results are shown in Fig. 8, which shows that, in the single-modal approaches (Only-Log and Only-Trace), the experimental results on the two datasets have a significant decrease compared to the three multi-modal approaches. This effectively illustrates the superiority of multi-modal data in detecting anomalies in microservice systems.

In the comparison experiment in which the nested graph structure was removed (No-NG), the F1-score decreased by 1.6% and 2.5% on the TT and GAIA datasets, respectively. This proves that using nested graphs in MSNGAD effectively captures the dependency relationships between log templates within spans. Due to the temporal and spatial dependencies between the generated log messages in the operation of the system, and the use of log template semantics only as a part of the fusion features cannot fully utilize the log features, thus ignoring some of the anomalies and leading to degraded detection results.

When comparing No-Aug and No-Cro, it was found that the F1-score remained stable across different datasets. This is because although No-Cro used the feature augmentation technique, it did not control the diversity of the generated pseudo-anomaly samples. In the diffusion reconstruction phase, the optimization goal of the diffusion reconstruction stage degrades to minimize the error between the generated sample features and the original input sample features. As training progressed, the generated samples and the original samples became very similar in feature space, which is equivalent to only increasing the number of anomaly samples without increasing the diversity of anomaly samples, and when the model encounter types of anomalies it has never seen before, false positives may still occur. In contrast, MSNGAD shows significant improvement in Recall compared to No-Cro, which further confirms the effectiveness of introducing contrastive learning to increase the diversity of anomalous samples. This enables the detection of more types of anomalies in the system, effectively reducing false positives.

5 Conclusion

In this paper, we propose MSNGAD, a microservice anomaly detection method based on nested graph diffusion reconstruction. It combines two types of monitoring data, traces and logs, and uses a unified nested graph representation to describe the complex structure of traces and the log

events embedded in them. MSNGAD uses GAT and GCN to extract features capturing the inner and outer graphs of nested graphs, respectively, and solves the problem of positive and negative sample imbalance in anomaly detection by feature augmentation using diffusion reconstruction method. We evaluated MSNGAD on two datasets and conducted experimental comparisons with single-modal and multi-modal methods, respectively, and the experimental results show that MSNGAD outperforms other anomaly detection methods and can effectively model the correlation between different data modalities as well as the temporal and spatial dependencies within each modality. In future work, we will continue to evaluate MSNGAD using different types of microservice systems to further explore the effectiveness and advantages of the methods in this paper.

Acknowledgements This work is supported by Liaoning Province Applied Basic Research Program Project (Grant No. 2023 JH2/101300195).

Author contributions All authors contributed to the study conception and design. Data collection, data processing, and analysis were performed by Xiuguo Zhang, Zhiying Cao, and Peipeng Wang. The first draft of the manuscript was written by Mengwei Fan and all authors commented on previous versions of the manuscript. All authors read and approved the final manuscript.

Data availability The data that support the findings of this study are available on request from the corresponding author.

Declarations

Ethical and informed consent for data used This study does not contain any studies with human participants or animals performed by any of the authors.

Conflict of interest The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

References

1. Opentracing.io: OpenTracing. (2021) https://opentracing.io/. Retrieved August 15, 2021
2. skywalking.apache.org: Apache SkyWalking. (2022) http://skywalking.apache.org/. Retrieved March 11, 2022
3. Zhang C, Wang X, Zhang H et al (2023) Layerlog: Log sequence anomaly detection based on hierarchical semantics. Appl Soft Comput 132:109860. https://doi.org/10.1016/j.asoc.2022.109860
4. Farzad A, Gulliver TA (2020) Unsupervised log message anomaly detection. ICT Express 6(3):229–237. https://doi.org/10.1016/j.icte.2020.06.003
5. Zhang C, Wang X, Zhang H et al (2021) Log sequence anomaly detection based on local information extraction and globally sparse transformer model. IEEE Trans Netw Serv Manage 18(4):4119–4133. https://doi.org/10.1109/TNSM.2021.3125967
6. Nedelkoski S, Cardoso J, Kao O (2019) Anomaly detection from system tracing data using multimodal deep learning. In: 2019 IEEE 12th International Conference on Cloud Computing (CLOUD). pp 179–186. https://doi.org/10.1109/CLOUD.2019.00038
7. Liu P, Xu H, Ouyang Q, et al. (2020) Unsupervised detection of microservice trace anomalies through service-level deep bayesian networks. In: 2020 IEEE 31st International Symposium on Software Reliability Engineering (ISSRE). pp 48–58. https://doi.org/10.1109/ISSRE5003.2020.00014
8. He Y, Zhang X, Wang P et al (2024) Microservice traces anomaly detection based on structural clustering and graph attention autoencoder. In: 2024 7th International Conference on Advanced Algorithms and Control Engineering (ICAACE). pp 977–981. https://doi.org/10.1109/ICAACE61206.2024.10549242
9. Zhang C, Peng X, Sha C et al (2022) Deeptralog: Trace-log combined microservice anomaly detection through graph-based deep learning. In: 2022 IEEE/ACM 44th International Conference on Software Engineering (ICSE). pp 623–634. https://doi.org/10.1145/3510003.3510180
10. Ying S, Wang B, Wang L, Li Q, Zhao Y, Shang J, Huang H, Cheng G, Yang Z, Geng J (2021) An improved knn-based efficient log anomaly detection method with automatically labeled samples. ACM Trans Knowl Discov Data 15(3):1–22
11. Zhang C, Peng X, Zhou T et al (2022) Tracecrl: contrastive representation learning for microservice trace analysis. In: Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering. ESEC/FSE 2022. Association for Computing Machinery, New York. pp 1221–1232. https://doi.org/10.1145/3540250.3549146
12. He P, Zhu J, Zheng Z et al (2017) Drain: an online log parsing approach with fixed depth tree. In: 2017 IEEE International Conference on Web Services (ICWS). pp 33–40. https://doi.org/10.1109/ICWS.2017.13
13. Brody S, Alon U, Yahav E (2021) How attentive are graph attention networks? arXiv:2105.14491
14. Kipf TN, Welling M (2016) Semi-supervised classification with graph convolutional networks. arXiv:1609.02907
15. Wang C, Zhang X, Gao H et al (2024) Colergs-constrained safe reinforcement learning for realising mass's risk-informed collision avoidance decision making. Knowl-Based Syst 300:112205. https://doi.org/10.1016/j.knosys.2024.112205
16. Kazemi MH, Alvanchi A (2025) Application of nlp-based models in automated detection of risky contract statements written in complex script system. Expert Syst Appl 259:125296
17. Liu Y, Wang H, Zhou H, Li M, Hou Y, Zhou S, Wang F, Hoetzlein R, Zhang R (2024) A review of reinforcement learning for natural language processing and applications in healthcare. J Am Med Inform Assoc 31(10):2379–2393
18. Santos CFGD, Arrais RR, Silva JVSd, Silva MHMd, Neto WBGdA, Lopes LT, Bileki GA, Lima IO, Rondon LB, Souza BMd et al (2025) Isp meets deep learning: a survey on deep learning methods for image signal processing. ACM Comput Survey 57(5):1–44
19. Xu C, Zhang T, Zhang D, Zhang D, Han J (2024) Deep generative adversarial reinforcement learning for semi-supervised segmentation of low-contrast and small objects in medical images. IEEE Trans Med Imaging 43(9):3072–3084. https://doi.org/10.1109/TMI.2024.3383716
20. Wang C, Zhang X, Gao H et al (2024) Optimizing anti-collision strategy for mass: a safe reinforcement learning approach to improve maritime traffic safety. Ocean Coast Manag 253:107161. https://doi.org/10.1016/j.ocecoaman.2024.107161
21. Du M, Li F, Zheng G et al (2017) Deeplog: anomaly detection and diagnosis from system logs through deep learning. In: Proceedings of the 2017 ACM SIGSAC conference on computer and communications security. pp 1285–1298

22. Meng W, Liu Y, Zhu Y et al (2019) Loganomaly: unsupervised detection of sequential and quantitative anomalies in unstructured logs. IJCAI 19:4739–4745
23. Zhang X, Xu Y, Lin Q et al (2019) Robust log-based anomaly detection on unstable log data. In: Proceedings of the 2019 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering. ESEC/FSE 2019. Association for Computing Machinery, New York, pp 807–817. https://doi.org/10.1145/3338906.3338931
24. Xiao R, Li W, Lu J, Jin S (2024) Contexlog: Non-parsing log anomaly detection with all information preservation and enhanced contextual representation. IEEE Trans Netw Service Manag
25. Wang P, Zhang X, Cao Z, Xu W, Li W (2024) Loggt: cross-system log anomaly detection via heterogeneous graph feature and transfer learning. Expert Syst Appl 251:124082
26. Xie Z, Xu H, Chen W et al (2023) Unsupervised anomaly detection on microservice traces through graph vae. In: Proceedings of the ACM Web Conference 2023. WWW '23. Association for Computing Machinery, New York, pp 2874–2884. https://doi.org/10.1145/3543507.3583215
27. Zhang K, Zhang C, Peng X et al (2022) Putracead: Trace anomaly detection with partial labels based on gnn and pu learning. In: 2022 IEEE 33rd International Symposium on Software Reliability Engineering (ISSRE). pp 239–250. https://doi.org/10.1109/ISSRE55969.2022.00032
28. Kong S, Ai J, Lu M, Gong Y (2024) Grand: Gan-based software runtime anomaly detection method using trace information. Neural Netw 169:365–377
29. Xie X, Jian S, Huang C (2023) Logtracead: anomaly detection from both logs and traces with graph representation learning. In: Proceedings of the 2023 2nd International Conference on Networks, Communications and Information Technology. CNCIT '23. Association for Computing Machinery, New York. https://doi.org/10.1145/3605801.3605824
30. Devlin J (2018) Bert: Pre-training of deep bidirectional transformers for language understanding. arXiv:1810.04805
31. Song X, Salcianu A, Song Y et al (2020) Fast wordpiece tokenization. Conference on empirical methods in natural language processing. Punta Cana, DOMINICAN REP, pp 07–11
32. Chen N, Liu Z, Hooi B, et al. (2024) Consistency training with learnable data augmentation for graph anomaly detection with limited supervision. In: The twelfth international conference on learning representations
33. You Y, Chen T, Sui Y et al (2020) Graph contrastive learning with augmentations. Adv Neural Inf Process Syst 33:5812–5823
34. Tian Y, Jiang A, Huang Q et al (2024) Latent diffusion-based data augmentation for continuous-time dynamic graph model. In: Proceedings of the 30th ACM SIGKDD conference on Knowledge Discovery and Data Mining. KDD '24, pp. 2900–2911. Association for Computing Machinery, New York. https://doi.org/10.1145/3637528.3671863
35. Schlegl T, Seeböck P, Waldstein SM, Schmidt-Erfurth U, Langs G (2017) Unsupervised anomaly detection with generative adversarial networks to guide marker discovery. CoRR. arXiv:1703.05921
36. Zhang H, Cisse M, Dauphin YN, Lopez-Paz D (2017) mixup: beyond empirical risk minimization. arXiv:1710.09412
37. Wang P, Zhang X, Cao Z (2025) Anomaly detection for microservice system via augmented multimodal data and hybrid graph representations. Inf Fusion 103017
38. Zhao N, Chen J, Yu Z et al (2021) Identifying bad software changes via multimodal anomaly detection for online service systems. In: Proceedings of the 29th ACM joint meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering. ESEC/FSE 2021. Association for Computing Machinery, New York, pp 527–539. https://doi.org/10.1145/3468264.3468543
39. Zhou X, Peng X, Xie T et al (2019) Latent error prediction and fault localization for microservice applications by learning from system trace logs. In: Proceedings of the 2019 27th ACM joint meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering. ESEC/FSE 2019. Association for Computing Machinery, New York, pp 683–694. https://doi.org/10.1145/3338906.3338961
40. Elastic (2022) Elasticsearch. Retrieved March 11, 2022 from https://www.elastic.co/elasticsearch/
41. Yu G, Chen P, Chen H et al (2021) Microrank: end-to-end latency issue localization with extended spectrum analysis in microservice environments. In: Proceedings of the Web Conference. pp 3087–3098
42. Meng Y, Zhang S, Sun Y et al (2020) Localizing failure root causes in a microservice through causality inference. In: 2020 IEEE/ACM 28th International Symposium on Quality of Service (IWQoS). pp 1–10. https://doi.org/10.1109/IWQoS49365.2020.9213058
43. Ma M, Xu J, Wang Y et al (2020) Automap: diagnose your microservice-based web applications automatically. In: Proceedings of The Web Conference 2020. WWW '20. Association for Computing Machinery, New York, pp 246–258. https://doi.org/10.1145/3366423.3380111
44. Mesh C (2022) Chaos Mesh. Retrieved March 11, 2022 from https://chaos-mesh.org/

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.